

APEXPLANET

**CYBERSECURITY & ETHICAL HACKING
INTERNSHIP PROGRAM**

TASK – 3

Web Application Security

Practical Task Report

Name	K R Sri Krishna Tulasi
ID	APSPL2643218
Course / Program	Cybersecurity & Ethical Hacking Internship Program
Date	17 September 2026
Task	Task – 3

1. Purpose

These notes summarize the recommended mitigation controls corresponding to the vulnerabilities demonstrated during Task 3. They are intended for the GitHub repository as concise technical documentation alongside the attack scenarios.

2. SQL Injection

Risk

SQL Injection occurs when untrusted input is incorporated into an SQL statement in a way that allows the input to alter query logic.

Primary Mitigation

Use prepared statements / parameterized queries. SQL code is defined separately from user-controlled values, preventing the value from being interpreted as SQL syntax.

Implementation Guidance

- Use parameterized queries for every database operation that incorporates user input.
- Validate input according to the expected data type and business rules.
- Use least-privilege database accounts so a compromised query cannot access unnecessary data.
- Do not expose detailed database errors to end users.
- Store passwords using an appropriate password-hashing algorithm rather than reversible or plaintext storage.

Verification

Repeat the malicious input after the fix. The application should treat the supplied value as data and should not return unintended records.

3. Cross-Site Scripting (XSS)

Risk

XSS occurs when attacker-controlled content is rendered as executable browser content instead of being safely handled as data.

Primary Mitigations

- Apply context-appropriate output encoding/escaping whenever untrusted data is rendered.
- Validate and constrain input where the application's requirements permit it.
- Deploy a restrictive Content Security Policy as defense in depth.
- Avoid unsafe DOM operations and APIs that interpret strings as HTML or script.

CSP Used in the Lab

The Apache configuration used the following policy:

```
default-src 'self'; script-src 'self'; object-src 'none'; base-uri 'self'
```

The policy was verified in the local HTTP response and used as an additional control during the XSS mitigation demonstration.

4. Cross-Site Request Forgery (CSRF)

Risk

CSRF can cause an authenticated browser to submit an unwanted state-changing request when the application does not adequately verify that the request was intentionally generated by the application.

Primary Mitigations

- Require a unique, unpredictable CSRF token for state-changing requests.
- Bind the token to the user's session and validate it server-side before processing the request.
- Use appropriate SameSite cookie settings as an additional browser-level defense.
- Where appropriate, validate request origin information as an additional control.
- Use POST for state-changing operations rather than relying on GET requests.

Verification

A tokenless request should be rejected. In the DVWA protected configuration, the demonstrated tokenless request was rejected with a CSRF-token error.

5. File Inclusion — LFI/RFI

Risk

File Inclusion vulnerabilities arise when user-controlled input is passed to file inclusion functions without sufficient restriction. Local File Inclusion may expose local files, while Remote File Inclusion can cause remotely supplied content to be included.

Recommended Mitigations

- Do not pass raw user-controlled values directly to include/require functions.
- Use an allowlist that maps safe application choices to fixed server-side filenames.
- Disable remote file inclusion functionality when it is not required.
- Restrict filesystem permissions and the application's access to sensitive files.
- Validate and normalize file-selection input before use.

The Task 3 practical requirement was to demonstrate LFI and RFI; mitigation was not a separate required File Inclusion deliverable, but these controls are recommended for production applications.

6. HTTP Security Headers

Security headers provide browser-enforced controls that complement application-level defenses.

Header	Configuration used	Purpose
Content-Security-Policy	default-src 'self'; script-src 'self'; object-src 'none'; base-uri 'self'	Restricts permitted content/script sources and adds defense against content injection.

X-Content-Type-Options	nosniff	Prevents browsers from MIME-sniffing responses.
X-Frame-Options	SAMEORIGIN	Restricts framing by other origins and helps mitigate clickjacking.
Referrer-Policy	strict-origin-when-cross-origin	Controls the amount of referrer information sent with requests.
Permissions-Policy	geolocation=(), microphone=(), camera=()	Restricts selected browser capabilities.

7. General Secure Development Practices

- Apply input validation and output encoding at appropriate trust boundaries.
- Use secure defaults and remove unnecessary services/features.
- Keep frameworks, libraries, PHP, Apache and operating-system components patched.
- Use centralized logging and monitor security-relevant errors.
- Test security controls again after changes to verify that the vulnerability is actually mitigated.

8. GitHub Use

These notes can be stored in the repository as mitigation-notes.md files under the relevant vulnerability folders. The formal Security Testing Report should remain the main evidence document, while the GitHub notes provide concise attack-and-mitigation documentation.